LIOR'S REALLY GOOD™ ROCKET OPTIMIZER

Optimizer Field Guide

Created by Lior Benshoshan

It takes an openMotor `.ric`, searches every buildable variation of it against your limits, and hands back motors you can open in openMotor and machine to the numbers. Here is how to drive it.

Start it

```
cd "Rocket Optimization"
python3 app.py                    # opens http://localhost:8420
```

That is the whole install. The first time, it says it needs to build an environment and waits for you to agree: a `.venv` in this folder, the packages in `requirements.txt`, and a pinned clone of openMotor. Nothing outside the folder is touched and your system Python is left alone. It does need **Python 3.9 to 3.12**, `git`, and a C compiler for openMotor's native extension — `xcode-select --install` on macOS, `build-essential` on Debian.

> **WHY 3.12 AND NOT NEWER**
>
> The pinned numpy, scipy and scikit-image publish wheels up to 3.12 only. PyPI does not record that as an upper bound, so on 3.13 pip does not refuse — it tries to compile numpy from source, which needs a C and Fortran toolchain and fails on most machines with a meson error that never mentions your Python version. Setup checks the range up front, and uses a supported interpreter if you have one installed alongside the newer one.

It loads whatever `.ric` is in `motor/` on start — drop yours in and it is the one that opens, whatever it is called. The folder starts empty and there is nothing to optimise until you put a motor there; `python tests/sample_motor.py motor/sample.ric` writes a throwaway one if you just want to try it. **Change** in the header picks a different file. Whatever you load is simulated exactly as the file says — the app never edits your motor behind your back.

THE SHORT VERSION

Load a motor → tick what may change and give each a step → say what you want more of → set your limits → press **Optimize** → pick a design off the curve → export the `.ric`. Everything below is detail on those seven moves.

The panels, in the order you use them

1

YOUR MOTOR

Check what you loaded

Grains, bore, nozzle, propellant, and what the motor does as it stands: thrust, impulse, peak pressure, Kn, mass flux. Warnings from openMotor appear here too — an over-expanded nozzle shows up as *low exit pressure*.

Read these numbers before anything else. If the motor as loaded already breaks a limit, the optimizer is not fixing a small problem, it is looking for a different motor.

2

FIXED HARDWARE

Confirm the case

Grain outer diameter, length and count are read from the file and shown greyed out. They are the tube and the mould — not optimizable, and not editable here. Change them in openMotor and reload.

Inhibited ends is the one control that *is* live, because it is a casting choice rather than hardware. If anything has been overridden, the panel says **modified from file** and offers **Reset to file**.

3

WHAT MAY CHANGE

Unlock dimensions and set your machining grid

One row per dimension: six grain cores, nozzle throat, nozzle exit, throat length. Untick a row to hold it at its current value; a held dimension leaves the search entirely rather than being searched over a zero-width range.

Step is your machining grid, and it is the field people skip. Type `0.01`, `1/16`, `1 1/16`, or `2-1/2` — shop fractions parse. Leave it blank and the optimizer will hand you a 2.4713″ core. The grid is anchored at zero, so a 1/16″ step offers whole sixteenths rather than sixteenths measured from wherever your lower bound happened to sit.

The chips below set every step at once. `0.01″` is two decimal places, which is the finest most people hold.

4

POSSIBLE CONFIGURATIONS

See how big the problem is

Updates live. It counts *distinct motors*, not grid points: cores are stored sorted, so one set of six diameters is one motor rather than 720. On the 5-inch motor with all nine dimensions free on a 0.01″ grid that is $\mathbf{4.5 \times 10^{19}}$, and brute force would take 4.4×10^{10} years.

With a Kn limit set, it also shows what the limits provably rule out and offers **Apply tighter bounds**. That narrows the search to a region where a legal motor can still exist — it never removes anything legal.

5

WHAT TO OPTIMIZE

Say what you want more of

Any of sixteen metrics, each set to maximize, minimize, or hit a target. Tick two and you get a trade-off curve instead of a single answer, which is almost always what you want — thrust and impulse pull against each other, and the interesting question is where on that curve you want to sit.

The presets cover the common cases: *Max initial thrust*, *Max impulse*, *Flattest curve*, *Thrust vs impulse*.

6

LIMITS IT MUST RESPECT

Set the envelope

Rows of *metric* ≤ or ≥ *value*. They start from your `.ric`'s own `maxPressure`, `maxMassFlux` and `minPortThroat`. Untick one to ignore it; add your own with **+ add**.

Nothing is reported unless it clears every ticked limit when re-simulated at full fidelity with all safety margins removed.

7

GRAIN CORES

Choose an ordering rule

Four options, and they change the answer more than people expect:

Any order	No rule. Rarely what you want — the aft grain ends up choking.
Never narrow aft	Cores never shrink going aft, ties allowed. The default, and what your current motor already does.
Strictly wider	Each core exceeds the one ahead by a minimum you set. Costs about 0.8 points of thrust versus allowing ties.
Paired mandrels	A few sizes shared across grains — 3 sizes × 2 grains, say. Fewer mandrels to cut.

8

HOW HARD TO LOOK

Set the budget and the number of searches

Quick / Standard / Thorough set a simulation budget (3,600 / 14,400 / 43,200), or type your own. **Independent searches** splits that budget across separate runs and merges what they find.

Raise the search count before the budget. Three 4,800-simulation searches merged beat a single 14,400-simulation search on the same motor — better answer, same cost.

Map the full trade-off trains surrogate models first. Slower, but it maps a denser curve. For most work the plain search is enough.

Reading the results

Four profiles across the top of the workspace. Same run, four questions.

ONE MOTOR

Design Review

Thrust curve, chamber pressure and Kn, a scaled cross-section, the spec sheet, and margin bars showing how close it runs to each limit. Amber means within 3% of a limit.

ALL OF THEM

Trade-off Explorer

The curve of options — click any point to load it into Design Review. Plus every design tried, a parallel-coordinates view of what the good ones have in common, and a table with a `.ric` button on every row.

THE SEARCH

Optimizer Diagnostics

Progress against simulations spent, which limit is actually binding, and — in trade-off mode — how accurate the models were and which dimension matters most.

BEFORE AND AFTER

Compare & Safety

Your motor against the selected design, a spec delta table, mass flux in each grain against its limit, and a tornado of what moves the leading objective.

WHICH LIMIT IS HOLDING YOU BACK

The bar chart in Diagnostics is the most useful panel in the app when a result disappoints. It shows what share of legal designs sit within 2% of each limit. A bar near zero means that limit is doing nothing; a red bar is the number to revisit.

Exporting and writing it up

Every row of the options table has a `.ric` button. The file opens in openMotor and re-simulates to exactly the numbers shown — nothing on screen is a model prediction.

The report writes the whole study up: hardware, limits, the trade-off curve, tabulated options and geometry, all derived from the run rather than boilerplate. A run that found nothing gets the most attention — which limit could never be met, how close anything came, and what would have to change.

Every run writes its own report — there is no button for it, because a report you have to remember to ask for is one you will not have. It lands as `reports/report.pdf`. **Download every design** gives you the other document: one sheet per design on the curve, with the numbers you would machine to, zipped so you can send the set. Meanwhile `outputs/` is emptied and rewritten with the run you just did — its result, its motors as `.ric`, its figures.

Five things worth understanding

`Initial thrust`	The mean of the thrust curve over the first 0.35 s, excluding openMotor's zero-thrust sample at t = 0. Not peak thrust, and not thrust at ignition.
`Legal`	Re-simulated at a 0.002 s timestep with every search-time margin removed, and clearing each ticked limit on *those* numbers.
`Kn and pressure`	One constraint wearing two hats — pressure is a monotone function of Kn. On this propellant 500 psi is reached at **Kn 224.5**, so a Kn ceiling of 225 is the looser of the two and raising it alone changes nothing.
`Seeds`	The search is stochastic. Identical runs have landed 6.4% apart on best thrust, which is why several run and merge by default. One search is a sample of the answer, not the answer.

`Grain order`	In openMotor's model, order changes nothing except mass flux and port/throat, and both are best with the widest core aft. So cores are always stored sorted, which removes a 720-fold redundancy from the search.

Worked example

The 5-inch motor: six 5.00 × 6.00 in BATES grains, Mad River Blue, uninhibited ends. As loaded it runs at 860 psi and Kn 323 — well outside a 500 psi / Kn 225 / 1.05 $lb{\cdot}in^{-2}s^{-1}$ envelope.

All nine dimensions free on a 0.01″ grid, both thrust and impulse maximized, 99,840 simulations across eight searches, 54 minutes:

#	CLASS	INITIAL N	IMPULSE N·S	ISP	PSI	KN	CORES (IN)	THROAT
1	N4165	7,194	11,429	181.9	497	224	3.58 3.84 4.00 4.11 4.24 4.48	1.75
2	N5315	6,944	17,730	196.0	495	223	3.37 3.44 3.49 3.52 3.60 3.89	1.72
3	O5200	6,700	22,164	197.5	497	224	2.97 2.99 3.05 3.06 3.19 3.38	1.68
4	O5459	6,455	25,014	197.8	497	224	2.59 2.76 2.76 2.78 2.79 2.98	1.65
5	O4788	6,190	26,995	197.5	493	222	2.33 2.36 2.42 2.43 2.62 2.89	1.63
6	O3976	5,859	28,462	194.8	494	223	1.82 1.84 1.90 2.33 2.58 2.80	1.60
7	O3441	5,550	29,141	191.9	478	218	1.44 1.47 1.87 2.14 2.44 2.70	1.60

Seven options off a 60-design front. Row 4 has the highest ISP and holds 90% of the best thrust with 86% of the best impulse.

> HOW GOOD IS 7,194 N?
>
> There is an arithmetic ceiling for this motor that no search can beat: bound one grain's burning area over the whole box, multiply by six, divide by the binding Kn to get the largest useful throat area, multiply by the best achievable thrust coefficient and the pressure limit. That gives **7,517 N**. The search got within 4.3% of a number nothing in the space can exceed.

Running it without the app

Everything the app does is a library call: `rocketopt.runner.run` takes a spec and a motor and hands back verified designs, and `rocketopt.report` and `rocketopt.bundle` turn those into the PDFs. The app is a thin layer over them, so anything it does can be scripted.

```
scripts/verify_ordering.py     check the core-sorting assumption against the simulator
.venv/bin/python -m pytest tests/ -q
```

> **IF A RUN COMES BACK EMPTY**
>
> That is a result, not a failure. Check Diagnostics for which limit was violated by everything, and read the report's *no answer* section — for a fixed nozzle it will tell you the throat diameter that would make a legal motor possible, and prove no core diameter could have.

---

openMotor 0.6.2 (GPLv3, vendored unmodified) · every number in this guide came from a simulation, not a model

App at `http://localhost:8420` · outputs under `outputs/`

Lior’s Really Good™ Rocket Optimizer · created by Lior Benshoshan